



Informe N°1, MCDI500 PROGRAMACIÓN PARA LA CIENCIA DE DATOS

Avance del proyecto, Fase 1 y 2 - Implementación inicial del entorno reproducible, exploración y procesamiento inicial de los datos del proyecto.

Docente: Dr. Omar Salinas Silva

Estudiantes: Matías Manriquez Ortiz, Daniel Pérez Ramirez, Abigail Robles Chávez,
Roberto Sánchez Saldivia.

Magíster en Ciencia de Datos e Inteligencia Artificial

Universidad Andrés Bello, Santiago.

Santiago, 23 de septiembre de 2026.



I. Índice	2
I. Índice	2
II. Introducción y contextualización	3
III. Definición de la problemática y objetivos del proyecto	5
Formulación de la problemática	5
Preguntas centrales del análisis	5
Objetivo general	6
Objetivos específicos	6
Alcance	7
Supuestos	7
Restricciones	7
Articulación con las fases F1-F4	8
IV. Aplicación de herramientas científicas.	9
Reproducibilidad técnica	10
Control de versiones	10
Documentación del proceso	11
V. Diseño de soluciones algorítmicas eficientes.	13
A. Codificación funcional	13
B. Preprocesamiento y transformación del dataset	14
C. Validación técnica y verificación del código.	15
Notebook F1_Definición.ipynb (F1)	16
Notebooks Ejecutables (F2)	17
Vinculación del mapa conceptual con el avance de proyecto.	22
Referencias	26
Anexos	27

II. Introducción y contextualización

El proyecto se formuló para trabajar con los datos de **Youth Risk Behavior Survey 2023**. La unidad de análisis son estudiantes entre 12 y 18 años que asisten a establecimientos públicos y privados de Estados Unidos. El archivo original contiene 20.103 registros y 117 variables, de las cuales se seleccionaron 11 columnas relacionadas con el uso de redes sociales, la salud mental, el sueño, la actividad física, las variables sociodemográficas y el diseño muestral. El subconjunto de trabajo conserva los mismos 20.103 registros: la reducción es de columnas, no de filas.

Esta investigación busca estimar la magnitud y dirección de la asociación entre la frecuencia de uso en redes sociales y la salud mental reportada en el YRBS 2023, mediante análisis ponderados. La adolescencia constituye una etapa especialmente relevante para este análisis, dado que en ella pueden manifestarse diversos problemas de salud mental. Sin embargo, tanto la exposición como el resultado de interés deben comprenderse considerando su complejidad. En este sentido Valkenburg et al. (2022) señalaron:

Tanto el uso de redes sociales como la salud mental son constructos sumamente complejos. Aunque la mayoría de los estudios se ha concentrado en la relación entre el uso de estas plataformas y la depresión o los síntomas depresivos, también deben investigarse plenamente los demás componentes de la salud mental, incluidos sus factores de riesgo. (p. 60, traducción propia)

El **alcance de este primer informe** corresponde a las Fases 1 y 2 del proyecto: la definición conceptual del problema, la configuración de un entorno reproducible y el

desarrollo del pipeline de obtención, exploración, limpieza, transformación y validación del conjunto de datos. El análisis estadístico más profundo, la programación de algoritmos estructurados/recursivos y la comunicación visual de resultados corresponden a las Fases 3 y 4, que se desarrollarán en las semanas siguientes.

La problemática requiere necesariamente un enfoque basado en programación y análisis de datos. El análisis considera horas de sueño, actividad física, sexo, edad, raza o etnicidad como variables de ajuste.

Cada fase del proyecto contribuye de forma acumulativa a responder la pregunta central:

- Fase 1: Define el problema y el entorno;
- Fase 2: Entrega un dataset limpio y validado;
- Fase 3: Aplica programación estructurada y recursiva para profundizar el análisis;
- Fase 4: Entrega los hallazgos mediante visualizaciones y un informe final.

III. Definición de la problemática y objetivos del proyecto

Formulación de la problemática

¿Qué relación existe entre los patrones de uso de tecnología y los indicadores de salud mental autopercebida y horas de sueño en los estudiantes incluidos en el dataset YRBS 2023, considerando variables sociodemográficas y de actividad física?

Esta es la pregunta central que orienta el proyecto, a partir de la cual se desprenden preguntas secundarias que guían el análisis exploratorio y, en fases posteriores, el análisis estadístico.

Preguntas centrales del análisis

- ¿Cuáles son las características sociodemográficas de la muestra y como se distribuyen los patrones de uso de tecnología, salud mental autopercebida y calidad del sueño?
- ¿Cuál es el estado de calidad del dataset YRBS 2023 en términos de valores perdidos, atípicos, duplicados e inconsistencias, y qué transformaciones de preprocesamiento se justifican para garantizar su validez analítica?
- ¿Existen patrones de asociación exploratorios observables entre el tiempo de uso de tecnología, la actividad física y la salud mental o el sueño?
- ¿De qué manera el uso conjunto de dispositivos, redes sociales, actividad física y factores sociodemográficos influye en la salud mental autopercebida al ser analizados mediante un modelo de regresión?

Objetivo general

Analizar la asociación entre la frecuencia de uso de redes sociales y la frecuencia de salud mental no buena reportada por estudiantes incluidos en el YRBS 2023, considerando las horas de sueño, la actividad física y las variables sociodemográficas, mediante un flujo de trabajo reproducible y documentado.

Objetivos específicos

- **Caracterizar** la distribución sociodemográfica y los patrones iniciales de uso de tecnología, salud mental y sueño en la muestra, mediante análisis exploratorio descriptivo para establecer la línea de base del estudio.
- **Diagnosticar y preprocesar** el conjunto de datos YRBS 2023 mediante un pipeline modular en Python, identificando y tratando valores faltantes, atípicos e inconsistencias para garantizar la integridad y calidad analítica de la data.
- **Validar** el dataset resultando mediante pruebas de consistencia técnica y verificación estructural con assert, asegurando la exportación de un conjunto limpio a data/processed/ para las fases posteriores.
- **Explorar y evaluar** las asociaciones preliminares entre las variables tecnológicas, de estilo de vida y salud mental, preparando la estructura de datos para el posterior modelamiento estadístico y regresional en las Fases 3 y 4.

Alcance

Este informe cubre exclusivamente las Fases 1 y 2: definición del problema, configuración del entorno reproducible y desarrollo del pipeline de datos (obtención, exploración, limpieza, transformación y validación). El análisis no incluye modelación predictiva avanzada, pruebas de hipótesis formales ni visualizaciones analíticas finales, los cuales se abordarán en las Fases 3 y 4.

Supuestos

El proyecto parte del supuesto operativo de que los registros son técnicamente utilizables para practicar procedimientos de ciencia de datos; sin embargo, no se asume que representen observaciones clínicas.

- Las variables de salud mental y horas de sueño declaradas por cada persona, se asumen como un reflejo válido de su estado percibido.
- Las relaciones identificadas serán interpretadas como asociaciones y no como causalidad.

Restricciones

- El análisis se restringe a las 11 variables originales del dataset; no se incorporan fuentes externas en esta etapa.
- La ausencia de información temporal impide analizar la evolución longitudinal.

- Diseño muestral complejo: el YRBS no es una muestra aleatoria simple; por eso incluye weight, stratum y psu. Como en las Fases 1 y 2 no se pondera, ninguna cifra descriptiva de este informe se generaliza a la población de estudiantes de EE. UU.: describe solo a los 20.103 encuestados. Las tres columnas se conservan en el dataset procesado para el análisis ponderado de las fases posteriores.

Articulación con las fases F1-F4

La problemática y los objetivos definidos en este apartado se desagregan operativamente en el notebook F1_Definicion.ipynb (Fase 1) y en el notebook S1_F2_Preprocesamiento.ipynb (Fase 2), que implementan respectivamente la configuración del entorno y el pipeline de datos descritos en las secciones siguientes.

En la Fase 3 se desarrollará la modelación y el análisis algorítmico del conjunto de datos, aplicando programación estructurada y funciones reutilizables para identificar y caracterizar las relaciones entre el uso de redes sociales, las horas de sueño, la actividad física y la salud mental autopercebida.

En la Fase 4 se abordará la comunicación e interpretación de los resultados mediante visualizaciones, síntesis de hallazgos, discusión de limitaciones y elaboración del informe final, permitiendo traducir los resultados técnicos en conclusiones comprensibles y fundamentadas.

IV. Aplicación de herramientas científicas.

El proyecto se trabajó en Python 3.14.7 con pandas 3.0.5 y NumPy 2.5.3, versiones se verifican en el apartado 6 del notebook S1_F1_Definicion.ipynb en la celda de ejecución contigua a este. Cada herramienta resuelve una necesidad concreta del proyecto: pandas y NumPy permiten cargar, seleccionar, diagnosticar y transformar de forma reproducible un archivo de 20.103 filas $\times$ 117 columnas; Jupyter integra en un mismo documento el código ejecutable, sus salidas y la narrativa que justifica cada decisión; y Git/GitHub registra quién, cuándo y por qué se modificó cada archivo del proyecto.[C6.1]

La lógica del pipeline no está escrita directamente en los notebooks, sino en el módulo `src/procesamiento.py`, que el notebook de la Fase 2 importa. El módulo contiene siete funciones de responsabilidad única (detalladas en el Anexo 3), con parámetros explícitos y anotaciones de tipo: `cargar_datos` (lee el CSV declarando `q6orig` como texto), `seleccionar_columnas` (reduce de 117 a 11 columnas por código), `diagnosticar_datos` (porcentaje de nulos, duplicados y tipos por columna), `clasificar_variables` (asigna el rol metodológico: identificador, muestral, nominal u ordinal), `transformar_datos` (convierte las escalas ordinales a categóricas ordenadas), `validar_datos` (comprueba columnas obligatorias, unicidad del identificador, duplicados y rangos) y `guardar_dataset` (exporta el CSV procesado, creando las carpetas si faltan).

Reproducibilidad técnica

La problemática requiere un enfoque computacional debido al volumen y variables de distinta naturaleza, incluyendo variables numéricas continuas, discretas y variables categóricas nominales.

El proyecto quedó con un único requirements.txt en la raíz, donde se fijaron las versiones usadas en el notebook de la Fase 1 (pandas 3.0.5, NumPy 2.5.3 e ipykernel). En el commit 5c357f9, del 15 de septiembre, se eliminaron los archivos requirements.txt duplicados que estaban en F1 y F2. En el README se dejaron las instrucciones para crear el entorno virtual .venv, instalar las dependencias y registrar el kernel "Python (grupo8-mcdi500)".

Control de versiones

El trabajo se registró en GitHub. Entre el 13 y el 23 de septiembre de 2026 se hicieron 51 commits, con aportes de los 4 integrantes. Inicialmente los integrantes trabajaron con sus cuentas personales, pero luego de consultarlo con el docente se empezaron a utilizar las cuentas institucionales, por lo cuál en GitHub se reflejan 8 entidades para los 4 integrantes del equipo. La mayoría de los mensajes siguió la convención de prefijos definida en el README (docs, data, feat, fix y test). Cinco commits hechos desde la interfaz web de GitHub quedaron con el mensaje por defecto. Se creó una rama secundaria, pero todos los cambios se integraron directamente en main, sin merges ni revisiones previas.

Documentación del proceso

La documentación se hizo en celdas markdown dentro de los notebooks. En el notebook de la Fase 1 se dejaron escritos el contexto del dataset, las preguntas de análisis, los objetivos, el alcance y los supuestos, junto con la verificación del entorno. El registro de cada transformación (dato original → problema detectado → criterio aplicado → resultado → validación) se aplica en el notebook de la Fase 2, que al cierre de este informe tiene definida su estructura. Además, en el archivo `bitacora_decisiones.md` se indica la Bitácora de decisiones técnicas que se tomaron correspondientes a las fases indicadas durante el proceso de trabajo.

```

proyecto-grupo8-mcdi500/
├── src/
│   └── procesamiento.py           funciones: carga, selección, diagnóstico,
│                                   clasificación, transformación, validación
├── F1/
│   ├── data/raw/
│   │   └── XXH2023_YRBSS_data.csv   dataset original YRBS 2023 (CDC)
│   └── notebooks/
│       └── S1_F1_Definicion.ipynb   Fase 1 – definición problema y
│                                   entorno
├── F2/
│   ├── data/processed/             salida del pipeline (entrada de F3)
│   │   └── yrbs2023_seleccion_procesada.csv   data procesada
│   └── notebooks/
│       └── S1_F2_Preprocesamiento.ipynb   Fase 2 – obtención, limpieza y
│                                   transformación
├── F3/
│   └── notebooks/
│       └── Fase 3.md                (pendiente: notebook de Fase 3)
├── F4/
│   └── notebooks/
│       └── Fase 4.md                (pendiente: notebook de Fase 4)
├── docs/
│   ├── bitacora_decisiones.md      registro de decisiones técnicas
│   ├── diccionario_variables.md    diccionario de variables data set
│   ├── Informe/
│   └── Mapa Conceptual Proyecto/
├── .gitignore
├── requirements.txt                dependencias del proyecto
└── README.md

```

V. Diseño de soluciones algorítmicas eficientes.

A. Codificación funcional

En la Fase 1 se definió el diseño del pipeline como una secuencia de funciones con parámetros explícitos (carga, diagnóstico, transformación y validación). Este diseño se implementó en el notebook de la Fase 2. En la Fase 1 el código se limitó a verificar el entorno y cargar el dataset con pandas, lo que confirmó 20.103 filas y 117 columnas. De esta forma, cada componente se diseñó para que pudiera probarse individualmente.

Por ejemplo: `def cargar_datos(ruta: str)`

```
def cargar_datos(ruta: str) -> pd.DataFrame:
    return pd.read_csv(
        ruta,
        dtype={"q6orig": "string"},
        low_memory=False,
    )
```

B. Preprocesamiento y transformación del dataset

En la revisión inicial de la Fase 1 en el dataset se verificó la estructura, compuesta por 20.103 registros y 11 variables, confirmando que el código base del entorno reproducible está funcionando. Siguiendo a esto, en la Fase 2 se revisan tipos de datos, identificadores duplicados, valores faltantes y códigos fuera de los dominios definidos. Las variables de edad, sexo, raza o etnicidad, salud mental, uso de redes sociales, sueño y actividad física se mantuvieron con sus códigos originales para preservar la trazabilidad, dejando recodificaciones o variables derivadas para los análisis posteriores.

Todas las transformaciones quedaron documentadas y se aplicaron sobre una copia procesada, manteniendo inalterado el archivo original.

Las cifras del preprocesamiento, obtenidas del notebook de la Fase 2, son las siguientes:

- Columna vacía: `orig_rec` tiene el 100 % de sus valores nulos (20.103 de 20.103).
- Casting: `q6orig` mezcla texto (por ejemplo, «N N») con códigos numéricos, lo que generaba un `DtypeWarning`; se declaró como string al cargarla, sin descartarla. Las cinco escalas ordinales (edad, actividad física, redes sociales, salud mental y sueño) se convirtieron a `pd.Categorical(ordered=True)`, lo que hace explícito su orden e impide operaciones aritméticas inválidas sobre los códigos.
- Duplicados: 0 filas duplicadas; `id_registro` es único en los 20.103 registros.
- Faltantes: edad 0,5 %, sexo 0,8 %, raza/etnicidad 1,8 %, actividad física 6,1 %, sueño 13,2 %, salud mental 21,9 % y redes sociales 24,4 %; las columnas de diseño

muestral no tienen faltantes. No se imputan: según el Apéndice C del codebook, ninguna de las siete variables depende de una pregunta previa, por lo que los nulos no se explican por salto de pregunta, e imputarlos introduciría respuestas que los estudiantes no entregaron.

- Normalización y escalamiento: no se aplican. Las variables de análisis son códigos ordinales o nominales, no magnitudes continuas; normalizarlas supondría distancias iguales entre categorías, que la escala no garantiza, peso_muestral tampoco se escala, porque es un factor de expansión y no una variable de análisis.
- Valores fuera de dominio: al tratarse de escalas con dominio cerrado, se verificó que todos los códigos estén dentro de su rango válido; validar_datos no encontró valores fuera de escala.
- Todas las transformaciones quedaron documentadas y se aplicaron sobre una copia procesada (yrbs2023_seleccion_procesada.csv), manteniendo inalterado el archivo original.

C. Validación técnica y verificación del código.

En la Fase 1 la verificación consiste en correr el notebook dentro del entorno y confirmar las versiones de Python, pandas y NumPy, además de la carga completa del dataset. En la Fase 2 se definen tres niveles de prueba para validar el pipeline:

- Casos normales: dataset completo con estructura esperada.
- Casos límite: data frame vacío, columnas con valores extremos o categorías inesperadas.

- Excepciones: archivo inexistente, columna obligatoria ausente o tipo incompatible.

Notebook *F1_Definición.ipynb* (F1)

En el cuaderno se detalla la información explicada anteriormente referente al contexto y formulación, preguntas de análisis, objetivo general y específicos; y alcance, restricciones y supuestos. Siguiendo a esto se indica la configuración y verificación del entorno, y el reconocimiento inicial del conjunto de datos:

▼ 6. Configuración y verificación del entorno

Código base que evidencia que el entorno reproducible está funcionando.

```
[1]: import sys
print('Python:', sys.version)

# Verificar librerías clave del entorno
import pandas as pd
import numpy as np
print('pandas:', pd.__version__)
print('numpy:', np.__version__)
```

```
Python: 3.14.7 | packaged by Anaconda, Inc. | (main, Aug 14 2026, 10:12:46) [MSC v.1942 64 bit (AMD64)]
pandas: 3.0.5
numpy: 2.5.3
```

Figura 1. Notebook F1, sección 6: verificación del entorno (versiones de Python, pandas y NumPy).

7. Reconocimiento inicial del conjunto de datos

Solo carga y una mirada superficial — sin limpiar ni transformar (eso es F2).

```
[2]: from pathlib import Path

RUTA = Path('../data/raw/XXH2023_YRBSS_data.csv') # completar: ruta al archivo del dataset
dtypes_problematizantes = {
    'q6orig': str,
    'q7orig': str,
    'q5': str
}

df = pd.read_csv(RUTA, dtype=dtypes_problematizantes)

print('Filas, columnas:', df.shape)
df.head()
```

Filas, columnas: (20103, 117)

	site	raceeth	q6orig	q7orig	record	orig_rec	q1	q2	q3	q4	...	q102	q103	q104	q105	q106	q107	BMIPCT	weight	stratum	psu
0	XX	NaN	505	180	1	NaN	3.0	1.0	1.0	NaN	...	2.0	3.0	3.0	2.0	2.0	2.0	97.083855	0.8614	103	16294
1	XX	5.0	N N	233	2	NaN	4.0	2.0	1.0	2.0	...	2.0	2.0	5.0	2.0	1.0	2.0	NaN	0.8920	103	16294
2	XX	5.0	506	165	3	NaN	5.0	2.0	3.0	2.0	...	1.0	2.0	4.0	2.0	2.0	1.0	92.262516	0.5081	103	16294
3	XX	5.0	N N	105	4	NaN	6.0	1.0	2.0	2.0	...	2.0	4.0	5.0	2.0	1.0	1.0	NaN	1.1759	103	16294
4	XX	5.0	601	125	5	NaN	3.0	2.0	1.0	2.0	...	2.0	2.0	5.0	2.0	2.0	1.0	7.567053	0.8920	103	16294

5 rows × 117 columns

Figura 2. Notebook F1, sección 7: carga inicial del archivo original (20.103 filas × 117 columnas).

Notebooks Ejecutables (F2)

En el cuaderno se detalla la información explicada anteriormente referente a la selección, transformación y validación en el conjunto de datos en base a la pregunta de investigación, detallando además hallazgos de calidad, selección de columnas con información relevante en base a las preguntas y objetivos, diagnóstico de variables faltantes y/o nulas, clasificación y transformación, validación y finalmente el almacenamiento del conjunto de datos procesado.

S1_F2_Preprocesamiento — Selección, transformación y validación

Dataset: 2023 Youth Risk Behavior Survey (YRBS), CDC — national (site=XX), 20.103 registros, 117 columnas originales.

Pregunta de investigación: ¿Qué relación existe entre los patrones de uso de tecnología y los indicadores de salud mental autopercebida y horas de sueño en los estudiantes incluidos en el dataset YRBS 2023, considerando variables sociodemográficas y de actividad física?

Fuente: Centers for Disease Control and Prevention (CDC). (2024). *2023 Youth Risk Behavior Survey Data* [conjunto de datos]. <https://www.cdc.gov/yrbs/data/index.html>

```
In [1]: import sys
from pathlib import Path

# Agrega src/ de la raíz del proyecto al path
sys.path.append(str(Path('.') / '..' / 'src'))

import pandas as pd
from procesamiento import (
    cargar_datos, seleccionar_columnas, diagnosticar_datos,
    clasificar_variables, transformar_datos, validar_datos,
    guardar_dataset, COLUMNAS_ANALISIS, COLUMNAS_SIN_DEPENDENCIA,
    ESCALAS_ORDINALES,
)

# Ruta relativa al notebook: F2/notebooks/ -> F1/data/raw/
RUTA_RAW = Path('.') / '..' / 'F1' / 'data' / 'raw' / 'XXH2023_YRBS5_data.csv'
df = cargar_datos(str(RUTA_RAW))
print('Dimensiones del archivo original:', df.shape)
```

Dimensiones del archivo original: (20103, 117)

Figura 3. Notebook F2: encabezado y carga del archivo original desde F1/data/raw/ (20.103 × 117).

1. Hallazgo de calidad: dtype mixto en q6orig

q6orig mezcla texto (p. ej. "N N") con códigos numéricos, lo que generaba un `DtypeWarning` al inferir el tipo automáticamente. Se resolvió declarando `dtype={'q6orig': 'string'}` en `cargar_datos`. No se descarta la columna: se documenta como un hallazgo de calidad real del archivo original.

```
In [2]: print(df['q6orig'].dtype)
df['q6orig'].value_counts(dropna=False).head(10)
```

string

```
Out[2]: q6orig
504    1686
506    1684
507    1633
505    1589
503    1552
508    1540
510    1365
509    1334
502    1256
511    1111
Name: count, dtype: int64[pyarrow]
```

2. orig_rec: columna 100% vacía

El validador genérico señaló `orig_rec` con más de 60% de nulos. En este dataset está vacía al 100%: se elimina.

```
In [3]: print('orig_rec: % nulos =', df['orig_rec'].isna().mean() * 100)
df = df.drop(columns=['orig_rec'])
print('Columnas tras eliminar orig_rec:', df.shape[1])
```

orig_rec: % nulos = 100.0

Columnas tras eliminar orig_rec: 116

Figura 4. Notebook F2, secciones 1 y 2: hallazgo de calidad en q6orig y eliminación de orig_rec (100 % nula).

3. Selección de columnas por código (desde el archivo original)

Con 117 columnas no es posible avanzar sin acotar. Se seleccionan, por código, únicamente las variables necesarias para responder la pregunta de investigación, declaradas explícitamente en `COLUMNAS_ANALISIS` (módulo `src/procesamiento.py`), con su código YRBS y su significado según el codebook oficial 2023.

```
In [4]: for col_original, col_nueva in COLUMNAS_ANALISIS.items():
        print(f'{col_original:>10} -> {col_nueva}')
```

```
record -> id_registro
weight -> peso_muestral
stratum -> estrato
psu -> psu
q1 -> edad_cod
q2 -> sexo_cod
raceeth -> raceeth_cod
q84 -> salud_mental_cod
q80 -> redes_sociales_cod
q85 -> sueno_cod
q76 -> actividad_fisica_cod
```

```
In [5]: df_sel = seleccionar_columnas(df)
        print('Dimensiones tras selección:', df_sel.shape)
        df_sel.head()
```

Dimensiones tras selección: (20103, 11)

```
Out[5]:
```

	id_registro	peso_muestral	estrato	psu	edad_cod	sexo_cod	raceeth_cod	salud_mental_cod	redes_sociales_cod	sueno_cod	actividad_fisica_cod
0	1	0.8614	103	16294	3.0	1.0	NaN	1.0	6.0	3.0	1.0
1	2	0.8920	103	16294	4.0	2.0	5.0	3.0	4.0	5.0	5.0
2	3	0.5081	103	16294	5.0	2.0	5.0	2.0	8.0	1.0	8.0
3	4	1.1759	103	16294	6.0	1.0	5.0	3.0	8.0	4.0	3.0
4	5	0.8920	103	16294	3.0	2.0	5.0	3.0	6.0	3.0	8.0

Figura 5. Notebook F2, sección 3: selección de 11 columnas por código (20.103 × 11).

4. Diagnóstico de faltantes en las variables seleccionadas

Las 7 variables de análisis (`q1`, `q2`, `raceeth`, `q76`, `q80`, `q84`, `q85`) **no tienen dependencia de otra pregunta** en el Apéndice C del codebook oficial (a diferencia, por ejemplo, de `Q34` que depende de haber fumado en `Q33`). Esto significa que, para este subconjunto, un valor nulo es **no responde genuino** (el estudiante vio la pregunta y la dejó en blanco), no "no aplica" estructural por salto de pregunta.

```
In [6]: diagnostico = diagnosticar_datos(df_sel)
        for k, v in diagnostico.items():
            print(f'{k}: {v}')
```

```
n_filas: 20103
n_columnas: 11
pct_nulos_por_columna: {'id_registro': np.float64(0.0), 'peso_muestral': np.float64(0.0), 'estrato': np.float64(0.0), 'psu': np.float64(0.0), 'edad_cod': np.float64(0.5), 'sexo_cod': np.float64(0.8), 'raceeth_cod': np.float64(1.8), 'salud_mental_cod': np.float64(21.9), 'redes_sociales_cod': np.float64(24.4), 'sueno_cod': np.float64(13.2), 'actividad_fisica_cod': np.float64(6.1)}
duplicados_totales: 0
tipos_de_datos: {'id_registro': 'int64', 'peso_muestral': 'float64', 'estrato': 'int64', 'psu': 'int64', 'edad_cod': 'float64', 'sexo_cod': 'float64', 'raceeth_cod': 'float64', 'salud_mental_cod': 'float64', 'redes_sociales_cod': 'float64', 'sueno_cod': 'float64', 'actividad_fisica_cod': 'float64'}
```

Figura 6. Notebook F2, sección 4: diagnóstico de faltantes, duplicados y tipos de datos.

5. Clasificación de variables: ordinal, no numérica

El validador genérico del curso clasificaría `q80`, `q84`, etc. como "discreta" por ser códigos enteros 1-8. Esa clasificación es incorrecta para este análisis: son escalas ordinales de frecuencia (p. ej. Q84: Never < Rarely < Sometimes < Most of the time < Always), y tratarlas como numéricas asumiría que la distancia entre "rara vez" y "a veces" es igual a la distancia entre "a veces" y "siempre", lo cual no está garantizado por el diseño de la escala.

```
In [7]:
clasificacion = clasificar_variables(df_sel)
for col, info in clasificacion.items():
    print(f"{col:22} -> {info['rol_equipo']}")
```

```
id_registro      -> identificador
peso_muestral    -> muestral (weight)
estrato          -> muestral (stratum)
psu              -> muestral (psu)
sexo_cod         -> nominal
raceeth_cod      -> nominal
edad_cod         -> ordinal
actividad_fisica_cod -> ordinal
redes_sociales_cod -> ordinal
salud_mental_cod -> ordinal
sueno_cod        -> ordinal
```

Figura 7. Notebook F2, sección 5: clasificación metodológica de las variables.

6. Transformación: tipado categórico ordenado

```
In [8]:
df_t = transformar_datos(df_sel)
df_t.dtypes
```

```
Out[8]: id_registro      int64
peso_muestral    float64
estrato          int64
psu              int64
edad_cod         category
sexo_cod         float64
raceeth_cod      float64
salud_mental_cod category
redes_sociales_cod category
sueno_cod        category
actividad_fisica_cod category
dtype: object
```

7. Decisión sobre ponderación (weight, stratum, psu)

Este dataset tiene diseño muestral complejo. **Decisión del equipo para las Fases 1-2: no se pondera.** Ponderar correctamente exige software de análisis de encuestas complejas (varianza por conglomerados), fuera del alcance de esta etapa. En consecuencia, **todo resultado descriptivo de este proyecto describe la muestra de 20.103 estudiantes encuestados en 2023, y no se generaliza a la población de estudiantes de EE. UU.** Las columnas `peso_muestral`, `estrato` y `psu` se conservan en el dataset procesado para un eventual análisis ponderado en fases posteriores.

Figura 8. Notebook F2, secciones 6 y 7: transformación a categórico ordenado y decisión de no ponderar en las Fases 1 y 2.

En la sección de validación, la función `validar_datos()` valida condiciones mínimas del DataFrame procesado: columnas obligatorias, unicidad de ID, sin duplicados y valores dentro del rango del codebook. Los resultados indican que toda la información pasa la validación, tal como lo indica la Figura 9. La evaluación técnica global retornó un estado `validacion_global: True`. El dataset procesado cumple con todos los criterios de integridad técnica y queda

oficialmente exportado en F2/data/processed/ como insumo validado para el análisis de asociación de la Fase 3

8. Validación

```
In [9]: resultado = validar_datos(df_t)
        for k, v in resultado.items():
            print(f'{k}: {v}')
```

```
columnas_obligatorias_presentes: True
id_registro_unico: True
sin_duplicados: True
edad_cod_en_rango: True
redes_sociales_cod_en_rango: True
salud_mental_cod_en_rango: True
sueno_cod_en_rango: True
actividad_fisica_cod_en_rango: True
validacion_global: True
```

9. Guardado del dataset procesado

```
In [10]: # Salida del pipeline: entrada directa de la Fase 3
        RUTA_SALIDA = Path('.') / 'data' / 'processed' / 'yrbs2023_seleccion_procesada.csv'
        ruta_salida = guardar_dataset(df_t, str(RUTA_SALIDA))
        print('Guardado en:', ruta_salida)
```

Guardado en: /Users/betosanchez/Desktop/Magister Data Science & IA /proyecto-grupo8-mcdi500/F2/data/processed/yrbs2023_seleccion_procesada.csv

Conclusión

El dataset se redujo de 117 a 11 columnas relevantes para la pregunta de investigación, seleccionadas por código y documentadas con su fuente en el codebook oficial. Se corrigió el hallazgo de calidad en `q6orig`, se eliminó `orig_rec` (vacía), se clasificaron las variables como ordinales (no numéricas), y se documentó la decisión de no ponderar junto con la limitación que eso implica. El dataset queda listo para el análisis de asociación en la Fase 3.

Figura 9. Notebook F2, secciones 8 y 9: validación (validacion_global: True), guardado del dataset procesado y conclusión.

Vinculación del mapa conceptual con el avance de proyecto.

La siguiente matriz (Tabla 1) vincula explícitamente cada bloque del mapa conceptual técnico elaborado en la actividad formativa de la Fase 1 con su materialización concreta en el repositorio, los notebooks, el README y este informe.

Elementos	Implementación en el proyecto	Evidencia verificable	Estado actual
1. Problema y preguntas	Definición del contexto, pregunta principal, objetivos y criterios de éxito del proyecto.	Celdas narrativas del notebook de la Fase 1, README e informe, particularmente en las secciones de definición del problema y objetivos.	Implementado
2. Datos	Selección e incorporación del 2023 National YRBS (CDC), 20.103 registros y 117 variables originales, de las cuales se seleccionan 11 por código. Se considera su almacenamiento como dato original y su posterior procesamiento.	Dataset original en F1/data/raw/XXH2023_YRBSS_data.csv; descripción de variables y codebook oficial referenciado en el notebook y en el informe. Los datos procesados se almacenan en F2/data/processed/.	Implementado
3. Entorno y análisis reproducible	Uso de Python, Jupyter Notebook y un entorno virtual .venv. Las librerías y dependencias necesarias para ejecutar el proyecto se registran en requirements.txt.	Entorno local configurado, notebooks ejecutables, archivo requirements.txt y celdas de importación y verificación de librerías.	Implementado

4. Control de versiones con Git	Uso de un repositorio local para registrar cambios mediante commits pequeños, frecuentes y descriptivos.	Historial de commits del repositorio, identificación de autores, fechas y cambios efectuados en los archivos del proyecto.	Implementado
5. GitHub y colaboración	Uso de un repositorio remoto compartido para respaldar, sincronizar e integrar el trabajo del equipo.	Repositorio grupal en GitHub, historial de contribuciones, sincronización de cambios y acceso colaborativo de los integrantes. El uso de ramas, Pull Requests y revisiones se aplicará según evolucione el desarrollo.	Implementado
6. Documentación científica en notebooks	Integración de código reproducible, narrativa explicativa, visualizaciones, referencias y resultados dentro de los notebooks.	Archivos .ipynb, celdas Markdown, comentarios de código, README, referencias incorporadas y secciones de resultados y conclusiones.	Implementado
7. Estructura actual del repositorio	Organización del proyecto mediante las carpetas F1/notebooks/ y F2/ (con datos y notebook de cada fase), F3/ y F4/ (pendientes), src/ (procesamiento.py), docs/ (mapa, informe, bitácora, validador) y Validar_DataSet.ipynb en la raíz, junto con requirements.txt y .gitignore.	Árbol de carpetas del repositorio real (verificable en GitHub), coincidente con el declarado en el README.	Implementado

8. Ejemplos de commits	Convención de prefijos (docs, data, feat, fix, test) definida en el README y aplicada en los commits reales del repositorio.	Ejemplos reales del historial: “docs: agrega mapa conceptual v1 y v2”; “data: incorpora dataset XXH2023_YRBSS_data.csv”; “feat: implementa funciones de preprocesamiento”; “fix: elimina requirements.txt duplicados en F1 y F2”.	Implementado
Limpieza, transformación y validación	Selección de 11 columnas por código, corrección del dtype de q6orig, eliminación de orig_rec, clasificación de variables ordinales y validación de columnas obligatorias, unicidad y rangos.	Notebook S1_F2_Preprocesamiento.ipynb ejecutado sin errores; dataset procesado en F2/data/processed/yrbs2023_seleccion_procesada.csv; resultado de Validar_DataSet.ipynb en docs/informe_dataset_validator.md.	En desarrollo
Modelación y análisis algorítmico	Aplicación de técnicas de programación y análisis orientadas a identificar relaciones entre el uso de tecnología y los indicadores de salud mental, estrés y calidad del sueño.	Código, funciones reutilizables, resultados analíticos y notebooks correspondientes a la Fase 3.	Proyectado
Comunicación final de resultados	Elaboración de visualizaciones interpretables, síntesis de hallazgos, conclusiones, limitaciones y recomendaciones.	Notebooks finales, visualizaciones, informe consolidado, README actualizado y productos de comunicación correspondientes a la Fase 4.	Proyectado

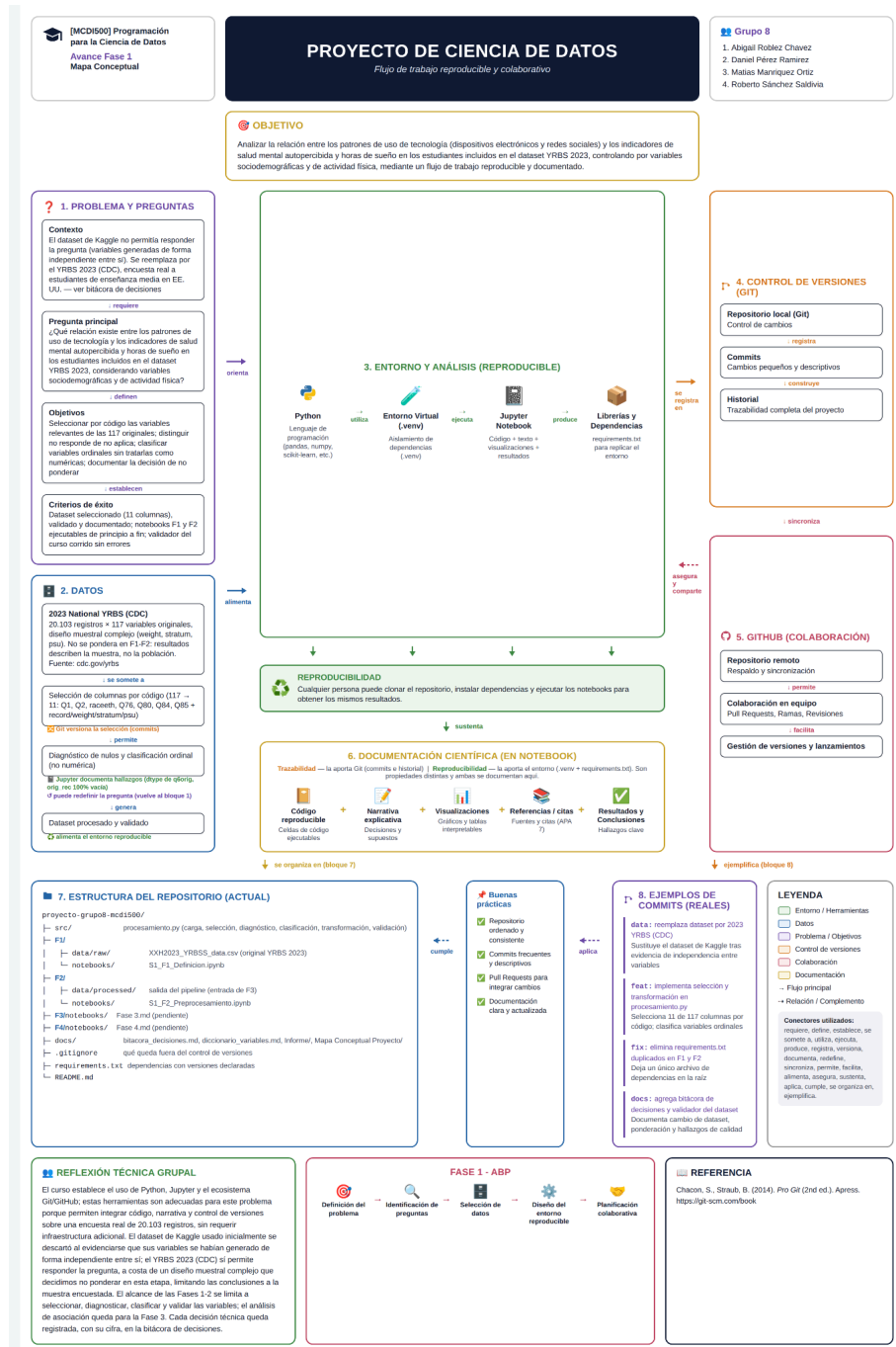
Tabla 1. Vinculación del mapa conceptual con el avance de proyecto.

Los bloques 1 a 8 del mapa conceptual quedaron materializados en el avance de las fases 1 y 2. La modelación y la comunicación visual de resultados, proyectadas en el mapa como trabajo de fases posteriores, se abordarán en las Fases 3 y 4 respectivamente, manteniendo coherencia entre la planificación conceptual y el desarrollo técnico real del proyecto. El mapa conceptual utilizado para la planificación inicial se presenta en el **Anexo 1**, con el fin de contrastar la estructura conceptual propuesta con el avance efectivo alcanzado en cada fase del proyecto.

Referencias

- Centers for Disease Control and Prevention. (2024). *2023 National Youth Risk Behavior Survey Data* [Conjunto de datos]. <https://www.cdc.gov/yrbs/data/index.html>
- Centers for Disease Control and Prevention. (2024). *2023 YRBS Data User's Guide* [Manual técnico y codebook].
<https://www.cdc.gov/yrbs/media/pdf/2023/2023-YRBS-SADC-Documentation.pdf>
- Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. <https://git-scm.com/book>
- NumPy Developers. (2025). *NumPy documentation*. <https://numpy.org/doc/stable/>
- The pandas development team. (2024). *pandas documentation* (Version 3.0.5) [Documentación técnica oficial]. <https://pandas.pydata.org/docs/>
- Universidad Andrés Bello, Facultad de Ingeniería. (2026a). *Guía de Git y GitHub*. Documento técnico de apoyo, MCDI500 Programación para la Ciencia de Datos, Magíster en Ciencia de Datos e Inteligencia Artificial.
- Universidad Andrés Bello, Facultad de Ingeniería. (2026b). *Sumativa 1: Avance del proyecto, Fases 1 y 2* [Pauta de evaluación]. MCDI500 Programación para la Ciencia de Datos.
- Valkenburg, P. M., Meier, A., & Beyens, I. (2022). *Social media use and its impact on adolescent mental health: An umbrella review of the evidence*. *Current Opinion in Psychology*, 44, 58–68. <https://doi.org/10.1016/j.copsyc.2021.08.017>
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>

Anexos



Anexo 1: Mapa conceptual v5 del proyecto — análisis del YRBS 2023

Anexo 2: Enlace al repositorio en github:

<https://github.com/MatiasIMO98/proyecto-grupo8-mcdi500>

Anexo 3: Constante y Fórmulas utilizadas - procesamiento.py

```
# =====
# CONSTANTES
# =====

COLUMNAS_ANALISIS = {
    "record": "id_registro",          # ID único de registro
    "weight": "peso_muestral",        # Factor de expansión CDC
    "stratum": "estrato",              # Estrato de diseño muestral
    "psu": "psu",                     # Unidad primaria de muestreo
    "q1": "edad_cod",                 # Edad (1 = ≤12 años ... 7 = ≥18 años)
    "q2": "sexo_cod",                 # Sexo (1 = Femenino, 2 = Masculino)
    "raceeth": "raceeth_cod",         # Raza/etnicidad (8 categorías)
    "q84": "salud_mental_cod",        # Salud mental percibida (1 = Never ... 5 = Always)
    "q80": "redes_sociales_cod",      # Horas/día en dispositivos electrónicos (1 = no usa ... 8
= ≥6 h)
    "q85": "sueno_cod",               # Horas de sueño (1 = ≤4 h ... 7 = ≥10 h)
    "q76": "actividad_fisica_cod",    # Días activo ≥60 min (1 = 0 días ... 8 = 7 días)
}

COLUMNAS_SIN_DEPENDENCIA = [
    "q1", "q2", "raceeth", "q76", "q80", "q84", "q85"
]

# Escalas ordinales: valores válidos ordenados de menor a mayor
ESCALAS_ORDINALES = {
    "edad_cod": [1, 2, 3, 4, 5, 6, 7],
    # 1 = ≤12 años, 2 = 13, 3 = 14, 4 = 15, 5 = 16, 6 = 17, 7 = ≥18 años

    "redes_sociales_cod": [1, 2, 3, 4, 5, 6, 7, 8],
    # 1 = no usa, 2 = <1 h, 3 = 1 h, 4 = 2 h, 5 = 3 h, 6 = 4 h, 7 = 5 h, 8 = ≥6 h diarias

    "salud_mental_cod": [1, 2, 3, 4, 5],
    # 1 = Never, 2 = Rarely, 3 = Sometimes, 4 = Most of the time, 5 = Always

    "sueno_cod": [1, 2, 3, 4, 5, 6, 7],
    # 1 = ≤4 h, 2 = 5 h, 3 = 6 h, 4 = 7 h, 5 = 8 h, 6 = 9 h, 7 = ≥10 h por noche

    "actividad_fisica_cod": [1, 2, 3, 4, 5, 6, 7, 8],
    # 1 = 0 días, 2 = 1 día, ..., 8 = 7 días (última semana)
}
```

```

# =====
# FUNCIONES
# =====

def cargar_datos(ruta: str) -> pd.DataFrame:
    """
    Carga el CSV del YRBS 2023 tal como viene del CDC, con una única
    corrección de tipo documentada.

    Corrección aplicada (bitácora 2.2):
    dtype={'q6orig': 'string'} -- resuelve el DtypeWarning causado por
    la mezcla de texto ("N N") y códigos numéricos en q6orig.
    La columna NO se descarta: la mezcla es un hallazgo de calidad
    real del archivo original y se documenta como tal.

    Nota: `orig_rec` se conserva en la carga para permitir su diagnóstico
    explícito en el notebook (bitácora 2.1). Su eliminación es una decisión
    del pipeline, no de la lectura del archivo.

    Args:
        ruta: ruta al CSV original (relativa al directorio de trabajo).

    Returns:
        DataFrame con 20.103 filas y 117 columnas.
    """
    return pd.read_csv(
        ruta,
        dtype={"q6orig": "string"},
        low_memory=False,
    )

def seleccionar_columnas(df: pd.DataFrame) -> pd.DataFrame:
    """
    Selecciona y renombra las columnas definidas en COLUMNAS_ANALISIS.

    La selección se hace por código desde el archivo original --
    nunca a mano -- para garantizar reproducibilidad y trazabilidad.

    Args:
        df: DataFrame completo (116 columnas).

    Returns:
        DataFrame con 11 columnas renombradas.
    """
    cols_presentes = [c for c in COLUMNAS_ANALISIS if c in df.columns]
    return df[cols_presentes].rename(columns=COLUMNAS_ANALISIS)

def diagnosticar_datos(df: pd.DataFrame) -> dict:

```

```

"""
Diagnóstico de calidad: nulos, duplicados y tipos de dato.

Args:
    df: DataFrame a diagnosticar.

Returns:
    dict con n_filas, n_columnas, pct_nulos_por_columna,
    duplicados_totales y tipos_de_dato.
"""
pct_nulos = {
    col: round(df[col].isna().mean() * 100, 1)
    for col in df.columns
}
return {
    "n_filas": len(df),
    "n_columnas": df.shape[1],
    "pct_nulos_por_columna": pct_nulos,
    "duplicados_totales": int(df.duplicated().sum()),
    "tipos_de_dato": {col: str(dtype) for col, dtype in df.dtypes.items()},
}

def clasificar_variables(df: pd.DataFrame) -> dict:
    """
    Clasifica cada variable según su rol metodológico (codebook 2023).

    ⚠ El validador automático del curso clasifica las ordinales como
    "discretas" porque sus valores son enteros 1-8. Esa clasificación
    es incorrecta: son escalas de frecuencia/intensidad ordenadas.
    Tratarlas como numéricas asumiría equidistancia entre categorías,
    lo cual el diseño de la escala no garantiza.

    Args:
        df: DataFrame con columnas renombradas.

    Returns:
        dict: columna → {'rol_equipo': str, 'nota': str}
    """
    mapa = {
        "id_registro": {"rol_equipo": "identificador", "nota": "ID único del encuestado"},
        "peso_muestral": {"rol_equipo": "muestral (weight)", "nota": "factor de expansión CDC"},
        "estrato": {"rol_equipo": "muestral (stratum)", "nota": "estrato de diseño muestral"},
        "psu": {"rol_equipo": "muestral (psu)", "nota": "unidad primaria de muestreo"},
        "sexo_cod": {"rol_equipo": "nominal", "nota": "1 = Femenino, 2 = Masculino"},
    }

```

```

        "raceeth_cod":          {"rol_equipo": "nominal",          "nota": "raza/etnicidad, 8
categorías"},
        "edad_cod":            {"rol_equipo": "ordinal",          "nota": "1 = ≤12 años ... 7
= ≥18 años"},
        "actividad_fisica_cod": {"rol_equipo": "ordinal",          "nota": "1 = 0 días ... 8 =
7 días/semana"},
        "redes_sociales_cod":  {"rol_equipo": "ordinal",          "nota": "1 = no usa ... 8 =
≥6 h/día"},
        "salud_mental_cod":     {"rol_equipo": "ordinal",          "nota": "1 = Never ... 5 =
Always"},
        "sueno_cod":           {"rol_equipo": "ordinal",          "nota": "1 = ≤4 h ... 7 =
≥10 h por noche"},
    }
    return {col: info for col, info in mapa.items() if col in df.columns}

def transformar_datos(df: pd.DataFrame) -> pd.DataFrame:
    """
    Convierte variables ordinales a pd.Categorical ordenado.

    Hace explícito el orden y evita que pandas las trate como
    numéricas continuas en operaciones de análisis posteriores.
    Las nominales (sexo_cod, raceeth_cod) se conservan como float64
    hasta la fase de modelado (F4).

    Args:
        df: DataFrame seleccionado y diagnosticado.

    Returns:
        DataFrame con tipos corregidos.
    """
    df_t = df.copy()
    for col, valores in ESCALAS_ORDINALES.items():
        if col in df_t.columns:
            df_t[col] = pd.Categorical(
                df_t[col],
                categories=valores,
                ordered=True,
            )
    return df_t

def validar_datos(df: pd.DataFrame) -> dict:
    """
    Valida condiciones mínimas del DataFrame procesado:
    columnas obligatorias, unicidad de ID, sin duplicados
    y valores dentro del rango del codebook.

    Args:
        df: DataFrame transformado.

```

```

Returns:
    dict con una clave por validación + 'validacion_global'.
"""
cols_obligatorias = set(COLUMNAS_ANALISIS.values())
resultado = {
    "columnas_obligatorias_presentes": cols_obligatorias.issubset(set(df.columns)),
    "id_registro_unico": (
        df["id_registro"].nunique() == len(df)
        if "id_registro" in df.columns else False
    ),
    "sin_duplicados": bool(df.duplicated().sum() == 0),
}
for col, valores_validos in ESCALAS_ORDINALES.items():
    if col in df.columns:
        vals = pd.to_numeric(df[col], errors="coerce").dropna()
        resultado[f"{col}_en_rango"] = bool(vals.isin(valores_validos).all())

resultado["validacion_global"] = all(resultado.values())
return resultado

def guardar_dataset(df: pd.DataFrame, ruta_relativa: str) -> Path:
    """
    Guarda el DataFrame procesado, creando directorios intermedios.

    Args:
        df: DataFrame procesado y validado.
        ruta_relativa: ruta relativa al directorio de trabajo del notebook.

    Returns:
        Path absoluto del archivo guardado.
    """
    ruta = Path(ruta_relativa)
    ruta.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(ruta, index=False)
    return ruta.resolve()

# Q5 admite respuestas múltiples de texto. raceeth combina Q4 y Q5 según CDC.
ESCALAS_NOMINALES = {"sexo_cod": [1, 2], "raceeth_cod": list(range(1, 9))}
ETIQUETAS_REDES = {1:"No usa", 2:"Unas veces al mes", 3:"Una vez a la semana",
    4:"Unas veces a la semana", 5:"Una vez al día", 6:"Varias veces al día",
    7:"Una vez por hora", 8:"Más de una vez por hora"}

def verificar_dominios(df):
    """Detecta códigos inválidos antes de que Categorical los oculte como NA."""
    errores = {}
    for col, permitidos in (**ESCALAS_ORDINALES, **ESCALAS_NOMINALES).items():
        if col not in df: raise ValueError(f"Falta columna {col}")

```

```
invalidos = df.loc[df[col].notna() & ~df[col].isin(permitidos), col]
errores[col] = {str(k):int(v) for k,v in invalidos.value_counts().items()}
return errores
```